

Product Requirement Document: RENTeasy vNext

Owner: Product

Version: vNext Planning Draft

Date: April 25, 2026

Objective

Scale RENTeasy from a functional prototype into a production-grade rental platform with:

1. Trust at transaction level (verification + moderation + dispute readiness).
2. Strong conversion and search relevance.
3. Integrated payments and financial controls.
4. Operational visibility and controlled growth loops.

Problem Statement

Current product proves core marketplace behavior, but growth and production readiness are limited by:

1. Mock auth and mock KYC.
2. No integrated payment/deposit/refund system.
3. Limited analytics instrumentation for funnel optimization.
4. Limited admin operations visibility in frontend.
5. Search misses due to lexical-only matching and no synonym intelligence.

User Types (vNext)

1. Renter: faster discovery, safer booking, transparent lifecycle.
2. Lender: better demand visibility, higher conversion, confidence in counterparty quality.
3. Admin/Ops: real controls for moderation, policy execution, and dispute handling.
4. Finance/Ops: reconciled source-of-truth for money movement and payouts.

Product Goals (90-120 days)

1. Increase search-to-request conversion by improving relevance and empty-result handling.
2. Introduce online payment and deposit flow with clear settlement lifecycle.
3. Reduce request abandonment with lifecycle nudges and SLA guardrails.
4. Enable admin frontend for moderation/KYC/report workflows.
5. Instrument full funnel with product analytics and operational metrics.

Non-Goals (for this vNext window)

1. Multi-country expansion.
2. Full AI chatbot/reels ecosystem.
3. Marketplace cross-border logistics.

Scope

Track A: Trust and Safety Foundation

1. Real KYC integration (replace mock status updates).
2. Value-threshold trust rules (mandatory checks by item value/risk level).
3. Moderation queue UI + audit trail.
4. Report listing/user with backend persistence and triage state.

Track B: Search and Conversion

1. Search relevance improvement:
 1. tokenization + basic synonyms dictionary.
 2. weighted scoring across title/subcategory/specifications.
2. Empty-state conversion:
 1. stronger "Request Item" CTA when no listings found.
 2. prefilled request wizard from failed search terms.
3. Better sort and ranking controls:
 1. relevance
 2. newest
 3. price low-high / high-low

Track C: Money Flow

1. Payment rails for rent and deposit authorization/capture.
2. Booking state integration with payment states.
3. Refund/cancellation policy enforcement.
4. Settlement and payout orchestration primitives.

Track D: Ops and Admin

1. Activate admin frontend routes.
2. Admin dashboards for:
 1. user/KYC moderation
 2. listing moderation
 3. request lifecycle health
 4. reported entities queue
3. Operational override controls with reason logging.

Track E: Analytics and Experimentation

1. Event taxonomy and SDK instrumentation.
2. Funnel dashboards:
 1. search -> listing -> request -> accepted -> completed
3. A/B hooks for search ranking and CTA variants.

System Specifications (vNext)

Architecture additions

```
`Frontend (Next.js)`  
`-> Backend API`  
`-> MongoDB`  
`+ Payment Gateway Integration`  
`+ KYC Provider Integration`  
`+ Analytics Pipeline (events -> warehouse/dashboard)`  
`+ Notification Service (email/sms/push, phased)`
```

Domain model additions (proposed)

1. `PaymentIntent` / `Transaction` entities.
2. `Dispute` entity with evidence and status timeline.
3. `Report` entity persisted (currently placeholder in admin response).
4. `AuditLog` for admin and ops actions.

Feature Specifications

1) Search Relevance v2

User flow

```
`Query input -> normalized query -> weighted retrieval -> ranked results -> filters/sorts -> click/listing request/open request`
```

Requirements

1. Query normalization for casing/plurals/common aliases.
2. Synonym map for frequent item terms.
3. Weighted matching:
 1. title > subcategory > category > specifications > description.
4. Highlight and explain relevance where possible.

Success metrics

1. +20% search-to-listing-click rate.
2. +15% search-to-request conversion.
3. Reduced "no results" rate.

2) Empty State to Demand Capture

User flow

```
`No results screen -> contextual CTA -> request wizard prefilled with query/city -> open request created`
```

Requirements

1. Prominent full-width request CTA when result count is zero.
2. Auto-fill request wizard with search phrase and selected location.
3. Track conversion from empty state to request post.

3) Payments and Deposits

User flow

`Request accepted -> payment intent created -> renter pays rent/deposit -> booking confirmed -> active -> completion -> release/refund`

Requirements

1. Payment intent lifecycle integrated with request status transitions.
2. Retry-safe webhook handling.
3. Cancellation windows and automated refund logic.
4. Failure-state UX for partial payment failures.

Success metrics

1. Payment success rate > 90%.
2. Failed-payment recoveries via retry.

4) Admin Console Activation

User flow

`Admin login -> admin dashboard -> moderation queue -> action -> audit log`

Requirements

1. Activate `/admin` route with role-guard.
2. Connect existing admin APIs.
3. Add filters, pagination, and reason capture for actions.

Success metrics

1. Moderation turnaround time.
2. Reduced unresolved report backlog.

5) Real KYC Integration

User flow

`KYC start -> provider session -> callback/webhook -> verified/pending/failed up date`

Requirements

1. provider token/session initiation endpoint.
2. webhook verification and signature checks.
3. user-level KYC state machine with immutable event history.

Success metrics

1. KYC completion rate.
2. Verification turnaround time.

Data & Events (must-have)

1. `search\_submitted`
2. `search\_result\_viewed`
3. `listing\_viewed`
4. `request\_started`
5. `request\_submitted`
6. `lender\_response\_sent`
7. `chat\_started`
8. `status\_transitioned`
9. `payment\_initiated`
10. `payment\_succeeded/failed`
11. `booking\_completed`

Constraints

1. Must preserve current API contracts wherever possible.
2. Progressive rollout required for payments and KYC.

3. Security and fraud controls must be considered before full scale.
4. Performance target should maintain current UX responsiveness.

Risks and Mitigations

1. Payment failure complexity.
 1. Mitigation: retries, idempotency keys, robust webhook reconciliation.
2. Increased moderation load.
 1. Mitigation: prioritized queues and lightweight automation rules.
3. Search ranking regressions.
 1. Mitigation: offline evaluations + A/B rollout.
4. Integration delays (KYC/payment vendors).
 1. Mitigation: adapter interface + fallback mock mode in staging.

Phased Delivery Plan

Phase 1 (Weeks 1-3)

1. Search relevance v2 baseline.
2. Empty-state request conversion.
3. Analytics event foundation.

Phase 2 (Weeks 4-7)

1. Admin frontend activation.
2. Report + moderation workflows.
3. KYC integration (start + callback path).

Phase 3 (Weeks 8-12)

1. Payment/deposit integration.
2. Refund/cancellation automation.
3. Finance reconciliation baseline dashboards.

KPI Targets

1. Search-to-request conversion: +15-25%.
2. Request-to-accepted conversion: +10-15%.
3. Request-to-completed conversion: +8-12%.
4. Time-to-first-lender-response: -20%.
5. Empty-result drop-off: -30%.

Open Questions

1. Which payment gateway and KYC provider are final?
2. What are item-value thresholds for mandatory trust gates?
3. Should delivery fee be platform-managed in vNext or later?
4. What is the initial dispute SLA and compensation policy?
5. Which analytics stack is preferred (self-hosted vs managed)?

Design Link

TBD (recommended to lock one canonical Figma source before implementation sprint).